

1 Key Concepts

1.1 DP Dimensionality

Last week, we saw several examples of dynamic programming problems with a single variable of concern (index of a list, the length of a string, etc.). However, some problems often involve multiple independent variables representing the possible scenarios in which we can find ourselves. For example, a substring can be defined as the portion of the string between two indices i and j . The dimensionality of the recurrence affects how the recurrence will be defined, and how the table will be implemented.

1.2 Shortest-Path DP

In this course, we consider the following two *shortest-path problems* on graphs. In both problems, the input is a weighted directed graph $G = (V, E)$, with a “weight function” $w: E \rightarrow \mathbb{R}$ that maps each edge to a weight (or cost, or length). Although individual edge weights can be negative, we assume that there is **no negative-weight cycle** in G . So, if there is a path from s to t in G , then there is a path with minimum total weight (or length); we call this a *shortest* path from s to t . We define $d(s, t)$ to be the total weight (or length) of the edges in such a path, and define $d(s, t) = \infty$ if no such path exists.

- **Single-Source Shortest Paths (SSSP):** Given a “source” vertex $s \in V$, find a shortest path from s to each vertex $t \in V$. (Value version: given s , find $d(s, t)$ for every $t \in V$.)
- **All-Pairs Shortest Paths (APSP):** For *every* pair of vertices $s, t \in V$, find a shortest path from s to t . (Value version: find $d(s, t)$ for every $s, t \in V$.)

Key observations.

1. If a shortest path P from s to t goes through vertex v , then P must be a **shortest path from s to v** , followed by a **shortest path from v to t** .

This optimal-substructure property can be shown by a substitution argument. Namely: if there were a shorter path from s to v , then we could substitute it for the s -to- v part of P , which would yield an s -to- t path that is even shorter than P —but this would contradict the hypothesis that P is a shortest such path.

2. Since there is no negative-weight cycle in the graph, there is a shortest path from s to t with **no cycle** in it. So, when considering a shortest path (from any source to any destination), we can assume without loss of generality that the path does not have a cycle.

SSSP recurrence (Bellman-Ford). Define $d^{(i)}(s, v)$ to be the length of a shortest path from s to v that uses exactly i edges, or ∞ if there is no such path. Then

$$d(s, v) = \min_{0 \leq i < n} d^{(i)}(s, v)$$

for each $v \in V$. This is because, by [Item 2](#), if there is an s -to- v path at all, then there is a shortest such path having less than n edges (since any path with n or more edges must visit some vertex more than once, and hence have a cycle). So, to compute $d(s, v)$ it suffices to compute the values of $d^{(i)}(s, v)$ for $0 \leq i < n$.

Due to [Item 1](#), the $d^{(i)}$ functions satisfy the following recurrence:

$$d^{(0)}(s, v) = \begin{cases} 0 & \text{if } s = v \\ \infty & \text{if } s \neq v. \end{cases}$$

$$d^{(i)}(s, v) = \min_{(u, v) \in E} \left\{ d^{(i-1)}(s, u) + w(u, v) \right\}.$$

APSP Recurrence (Floyd-Warshall). Fix an arbitrary ordering of the vertices, as $V = \{v_1, v_2, \dots, v_n\}$. Then define $d^{[i]}(s, v)$ to be the length of a shortest path from s to v **whose internal vertices (if any) are in** $\{v_1, \dots, v_i\}$; or ∞ if there is no such path. Then

$$d(s, v) = d^{[n]}(s, v)$$

for each $v \in V$. This is because in any shortest s -to- v path, its internal vertices are in $V = \{v_1, \dots, v_n\}$. In other words, the restriction on the internal vertices imposed by $d^{[i]}$ is not a restriction at all when $i = n$.

Due to [Items 1](#) and [2](#), the $d^{[i]}$ functions satisfy the following recurrence:

$$d^{[0]}(s, t) = \begin{cases} 0 & \text{if } s = t \\ w(s, t) & \text{if } (s, t) \in E \\ \infty & \text{otherwise.} \end{cases}$$

$$d^{[i]}(s, t) = \min \left\{ d^{[i-1]}(s, t), d^{[i-1]}(s, v_i) + d^{[i-1]}(v_i, t) \right\}.$$

1.3 Greedy Algorithms

A *greedy algorithm* computes a solution to an optimization problem by making (and committing to) a sequence of locally optimal choices. In general, there is no guarantee that such a sequence of locally optimal choices produces a global optimum. However, for some specific problems and greedy algorithms, we can prove that the result is indeed globally optimal.

Exchange arguments. A standard strategy for proving that a greedy algorithm correctly solves a particular optimization problem proceeds by an *exchange argument* (like the ones we gave for unweighted task selection, and for minimum spanning trees). Such an argument typically aims to show that the algorithm's output is optimal by first proving it is a subset of some optimal solution, then arguing they must be the same using the greedy algorithm's condition.

To show that the algorithm's output is a subset of some optimal solution, we proceed as follows:

1. Let $\text{ALG} = \{s_1, s_2, \dots, s_m\}$ denote the algorithm's final —i.e., its full ordered sequence of selected objects—and let OPT be some *arbitrary* optimal solution.
2. If ALG happens to be a subset of OPT , then we have reached the desired conclusion, and there is nothing more to prove. So, suppose otherwise, and let s_{diff} denote the first object selected by the algorithm that is not in OPT . (So, all the objects $s_1, \dots, s_{\text{diff}-1}$ that the algorithm selected prior to s_{diff} are in OPT .)
3. We then modify OPT into a different optimal solution OPT' that contains all of $s_1, \dots, s_{\text{diff}}$. We do this by identifying a suitable element $f \in \text{OPT}$ (where $f \notin \{s_1, s_2, \dots, s_{\text{diff}}\}$) and exchanging it with s_{diff} , defining $\text{OPT}' = \text{OPT} \cup \{s_{\text{diff}}\} \setminus \{f\}$.

4. We must choose the exchanged element f so that OPT' is also an optimal solution, i.e., it is valid (it meets all the constraints imposed by the problem), and it is optimal. The latter often follows because s_{diff} is “no worse than” f in terms of what we want to optimize. Arguing the former typically depends on the specifics of the problem and how f is defined, but it often uses the fact that s_{diff} is “compatible” with $s_1, \dots, s_{\text{diff}-1}$ (because all of them are in ALG).
5. Altogether, the exchange step shows that since the first $\text{diff} - 1$ selected objects from ALG belong to some optimal solution, then the first diff selected objects from ALG also belong to some (different) optimal solution. So, by repeated application of the exchange step (formally, using induction), we can conclude that the algorithm’s entire output ALG is a subset of some optimal solution.

Finally, it just remains to show that ALG is equal to (and not just a subset of) an optimal solution. This is typically done by arguing that ALG is maximal, i.e., that no more objects can be added to it without violating the constraints of the problem. So, ALG itself must be optimal.

2 Discussion Problems

1. Longest Palindromic Subsequence

Give a recurrence relation (including base cases) that is suitable for dynamic programming solutions to the following problem. You do not need to prove your correctness.

Given a string S , determine the length of a longest subsequence of S (the characters do not need to be consecutive) that is a palindrome. Recall that a palindrome is a string which is the same forwards and backwards (e.g., “racecar”).

2. Bellman-Ford and Floyd-Warshall.

Select **all the true statements**.

- ☐ If p is the *unique* shortest s -to- t path in a graph—i.e., every other s -to- t path has strictly larger length—and v is a vertex on p , then the s -to- v portion of p is the *unique* shortest s -to- v path in the graph.
- ☐ In the Floyd–Warshall recurrence, the values $d^{[i]}(s, t)$ are *non-increasing* as i increases, i.e., $d^{[j]}(s, t) \leq d^{[i]}(s, t)$ for all $j \geq i$ and all vertices s, t .
- ☐ If a graph has no negative-weight cycle, then no shortest path in the graph contains a cycle.
- ☐ To fill each individual entry of the DP table in the Floyd–Warshall algorithm, we need to look at only a constant number of cells, whereas this may not be the case in the Bellman–Ford algorithm.

3. Vertex-weighted single-source lowest-cost paths (VWSSLCP).

In this problem we consider a variation of shortest-paths DP. Suppose we are given a graph $G = (V, E)$ and a cost function $c: V \rightarrow \mathbb{R}$ that maps each *vertex* to a cost. The cost of a path is simply the total cost of all the vertices on the path (including the starting and ending vertex). Again, we assume that there is **no negative-cost cycle** (although individual vertex costs can still be negative).

Given a source vertex $s \in V$, our goal is to find the *minimum path cost* from s to each vertex $v \in V$. Give a recurrence relation for this problem. Remember to include appropriate base cases(s).

4. Copying homework problems.

You are copying n problems from your textbook to the sheets of paper you do your homework on. You need to copy all the problems, preserving their order. The problems fill various numbers of lines, and each sheet of paper has the same number of lines on it. Each problem must appear on a single sheet of paper; it can't be split across more than one sheet.

- (a) Describe a greedy algorithm to minimize the number of pages you need.

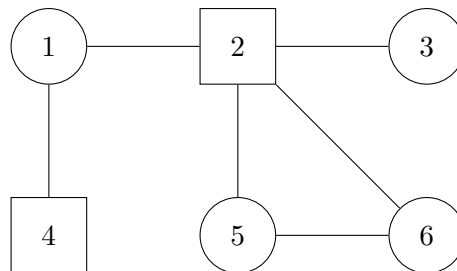
- (b) Prove that the proposed greedy algorithm is correct. In other words, prove that the proposed algorithm uses the minimum number of pages, while satisfying the constraints given in the problem.

5. Dominating Set.

A *dominating set* S in a graph G is a set of vertices for which every vertex of G either is in S , or is adjacent to some vertex in S .

We are interested in a *smallest* dominating set of a given graph, i.e., one that has the fewest possible vertices. (There may be more than one smallest dominating set.)

For example, the following graph has a smallest dominating set $S^* = \{2, 4\}$: every vertex other than 2 and 4 is adjacent to 2 or 4 (or both), and there is no dominating set consisting of a single vertex.



Consider the following greedy algorithm for finding a dominating set in a graph.

```

1: function GREEDYDS( $G$ )
2:    $S \leftarrow \emptyset$ 
3:   while  $G$  has at least one vertex do
4:     Select any vertex  $v$  in  $G$  that has largest degree (i.e., the most neighbors)
5:     Add  $v$  to  $S$ 
6:     Remove  $v$  and all its neighbors, including all incident edges, from  $G$ 
7:   return  $S$ 

```

Give a small graph G on which the algorithm **might not return a *smallest* dominating set**. Specifically, **give a sequence of vertices that the algorithm might choose** to make up its final output set, and **give an optimal dominating set of G that is smaller than this output set**.

3 Additional Problems

1. Matrix-chain multiplication

Given a sequence of matrices $(M[1, \dots, n])$ to multiply together, determine the minimum number of element-element multiplications required (you may disregard additions).

As an example, suppose the sequence of matrices to multiply is ABC , where A is 2×3 , B is 3×4 , and C is 4×5 .

Computing $(AB)C$ requires $2 \cdot 3 \cdot 4 + 2 \cdot 4 \cdot 5 = 64$ multiplications. Computing $A(BC)$ requires $3 \cdot 4 \cdot 5 + 2 \cdot 3 \cdot 5 = 90$ multiplications. Therefore, the minimum number of multiplications required is 64.

The following is a short example of how to perform matrix multiplication:

If $A = \begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}$ and $B = \begin{bmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{bmatrix}$, then $AB = \begin{bmatrix} a_{00}b_{00} + a_{01}b_{10} & a_{00}b_{01} + a_{01}b_{11} \\ a_{10}b_{00} + a_{11}b_{10} & a_{10}b_{01} + a_{11}b_{11} \end{bmatrix}$.

Remarks: Matrix multiplication is associative. Given two matrices with dimensions $m \times n$ and $n \times p$, we require mnp multiplications because the resulting matrix is of dimension $m \times p$ and each element requires n multiplications. You can read more about matrix multiplication here: https://en.wikipedia.org/wiki/Matrix_multiplication

2. Scooby-Doo escape room.

Scooby-Doo is stuck in an $n \times n$ -sized escape room; the “northwest” corner of the room is denoted $(1, 1)$. Each 1×1 square in this room has a certain amount of Scooby-Snacks. You are given the matrix H , where $H[i][j] := H_{ij}$ contains the number of Scooby-Snacks in the (i, j) th square of the room.

Scooby wants to maximize the number of Scooby Snacks he eats while escaping from a monster coming from the north side of the room. If he is at square (i, j) , then from his current location he can move only to squares $(i + 1, j)$, $(i + 1, j + 1)$ or $(i + 1, j - 1)$. (In other words, he can only move directly south, southeast, or southwest.) Otherwise, the monster will catch up and eat him!

Scooby escapes when he reaches the south wall i.e. a location of the form $(n, \star)$ (in other words when Scooby gets to the other end of the room). Scooby can start from any square $(1, \star)$ in the northernmost row.

- (a) Give a recurrence relation (including base cases), that would yield an $O(n^2)$ time dynamic programming solution to the problem. Briefly explain why your recurrence relation is correct.

- (b) Give pseudocode implementing an $O(n^2)$ dynamic programming algorithm MAX-SCOOPY-SNACK that computes the maximum number of Scooby Snacks that Scooby can eat during his escape under the problem constraints.